Smart Contract
Security Audit Report

The SlowMist Security Team received the edgeX team's application for smart contract security audit of the EdgeToken (EDGE) on 2026.03.19. The following are the details and results of this smart contract security audit:

Token Name :

EdgeToken (EDGE)

The contract address :

Proxy:

https://etherscan.io/token/0xB0076DE78Dc50581770BBa1D211dDc0aD4F2a241

Implementation:

https://etherscan.io/address/0x9406dd76452c172b3cc77092b42398e87dab3161

The audit items and results :

(Other unknown security vulnerabilities are not included in the audit responsibility scope)

NO.	Audit Items	Result
1	Replay Vulnerability	Passed
2	Denial of Service Vulnerability	Passed
3	Race Conditions Vulnerability	Passed
4	Authority Control Vulnerability Audit	Passed
5	Integer Overflow and Underflow Vulnerability	Passed
6	Gas Optimization Audit	Passed
7	Design Logic Audit	Passed
8	Uninitialized Storage Pointers Vulnerability	Passed
9	Arithmetic Accuracy Deviation Vulnerability	Passed
10	"False top-up" Vulnerability	Passed
11	Malicious Event Log Audit	Passed
12	Scoping and Declarations Audit	Passed

NO.	Audit Items	Result
13	Safety Design Audit	Passed
14	Non-privacy/Non-dark Coin Audit	Passed

Audit Result : Passed

Audit Number : 0X002603190001

Audit Date : 2026.03.19 - 2026.03.19

Audit Team : SlowMist Security Team

Summary conclusion : This is an ERC20 token contract that contains the Upgradeable sections. The total amount of contract tokens can be changed, users can burn their owner tokens through the burn and burnFrom functions. SafeMath security module is used, which is a recommended approach. The contract does not have the Overflow and the Race Conditions issue.

During the audit, we found the following information:

1. The UPGRADER_ROLE can upgrade the Implementation address through the upgradeTo and upgradeToAndCall functions.

The source code:

```
// SPDX-License-Identifier: MIT
//SlowMist// The contract does not have the Overflow and the Race Conditions issue
pragma solidity ^0.8.24;

import "@openzeppelin/contracts-upgradeable/proxy/utils/Initializable.sol";
import "@openzeppelin/contracts-upgradeable/token/ERC20/ERC20Upgradeable.sol";
import "@openzeppelin/contracts-
upgradeable/token/ERC20/extensions/ERC20BurnableUpgradeable.sol";
import "@openzeppelin/contracts-
upgradeable/token/ERC20/extensions/ERC20PermitUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/access/AccessControlUpgradeable.sol";
import "@openzeppelin/contracts-upgradeable/proxy/utils/UUPSUpgradeable.sol";

contract EdgeToken is
    Initializable,
    ERC20Upgradeable,
    ERC20BurnableUpgradeable,
    ERC20PermitUpgradeable,
    AccessControlUpgradeable,
```

```
    UUPSUpgradeable
{
    error InvalidAdmin();
    error ArrayLengthMismatch();
    error CapExceeded();
    error TotalMintedMismatch();

    bytes32 public constant UPGRADER_ROLE = keccak256("UPGRADER_ROLE");
    uint256 public constant MAX_SUPPLY = 1_000_000_000 * 10 ** 18;

    /// @custom:oz-upgrades-unsafe-allow constructor
    constructor() {
        _disableInitializers();
    }

    function initialize(
        address admin_,
        address[] memory initialRecipients,
        uint256[] memory initialAmounts
    ) public initializer {
        if (admin_ == address(0)) revert InvalidAdmin();
        if (initialRecipients.length != initialAmounts.length) revert
ArrayLengthMismatch();

        __ERC20_init("Edge", "EDGE");
        __ERC20Burnable_init();
        __ERC20Permit_init("Edge");
        __AccessControl_init();
        __UUPSUpgradeable_init();

        _grantRole(DEFAULT_ADMIN_ROLE, admin_);
        _grantRole(UPGRADER_ROLE, admin_);

        uint256 totalMinted;
        for (uint256 i = 0; i < initialRecipients.length; i++) {
            address to = initialRecipients[i];
            uint256 amount = initialAmounts[i];
            //SlowMist// A 0 address or a 0 amount parameter will be skipped in the
loop.
            if (to != address(0) && amount > 0) {
                totalMinted += amount;
                if (totalMinted > MAX_SUPPLY) revert CapExceeded();
                _mint(to, amount);
            }
        }

        if (totalMinted != MAX_SUPPLY) revert TotalMintedMismatch();
    }
```

```
    // decimals() defaults to 18 from ERC20Upgradeable
    //SlowMist// The UPGRADER_ROLE can upgrade the Implementation address through the
upgradeTo and upgradeToAndCall functions
    function _authorizeUpgrade(address newImplementation)
        internal
        override
        onlyRole(UPGRADER_ROLE)
    {}

    uint256[50] private __gap;
}
```

Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.

Official Website

www.slowmist.com

E-mail

team@slowmist.com

Twitter

@SlowMist_Team

Github

https://github.com/slowmist